

- This lab will cover asymptotic analysis, and problem solving.
- It is assumed that you have reviewed chapter 3 of the textbook. You may want to refer to the text and your lecture notes during the lab as you solve the problems.
- When approaching the problems, think before you code. Doing so is good practice and can help you lay out possible solutions.
- Think of any possible test cases that can potentially cause your solution to fail!
- If you finish early, you may help other students or leave early after showing your work to the TA. If you don't finish by the end of the lab, we recommend you complete it on your own time. Ideally, you should not spend more time than suggested for each problem.
- Your TAs are available to answer questions in the lab, during office hours, and on Ed Discussion.

Vitamins (60 minutes)

For **big-O proofs**, you must show that there exists constants c , and n_0 such that $f(n) \leq c \cdot g(n)$ for every $n \geq n_0$ in order to conclude $f(n) = O(g(n))$.

For **big- θ proofs**, you must show that there exists constants c_1 , c_2 , and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for every $n \geq n_0$ in order to conclude $f(n) = \theta(g(n))$.

1. Use the **formal proof of big-O and big- θ** in order to show the following (10 minutes):

a. $n^2 + 5n - 2$ is $O(n^3)$

b. $\frac{n^2 - 1}{n + 1}$ is $O(n)$

c. $\sqrt{36n^2 - 72n + 36}$ is $\theta(n)$

d. $4^{\log_2 n}$ is $\theta(n^2)$

2. State **True** or **False** and explain why for the following (10 minutes):

a. $8n^2(\sqrt{n})$ is $O(n^3)$

b. $8n^2(\sqrt{n})$ is $\theta(n^3)$

3. Given the generator function, write the output: (10 minutes)

```
def mystery(n):  
    total = 0  
  
    for i in range(1, n + 1):  
        total += i  
  
        if i % 2 == 0:  
            yield total  
  
        total -= 1  
  
for x in mystery(8):  
    print(x, end=" ", )
```

>>> _____

4. Sort the following functions in an increasing asymptotic order and write $<$ or $\leq$ between each two subsequent functions to indicate if the first is asymptotically less than or asymptotically equivalent to the second function respectively. **For answer-checking purposes, your order should be the numbered functions, not their definitions (i.e.**

f_1, f_2, f_3 , **etc, not n, n^2, n^3 , etc.**).

For example, if you were to sort: $f_1(n) = n$, $f_2(n) = \log(n)$, $f_3(n) = 3n$, $f_4(n) = n^2$,
your answer could be $\log(n) < (n \leq 3n) < n^2$

Hint: Try grouping the functions like so: linear, quadratic, cubic, exponential ... etc

- a. (functions f_1 through f_{15})

$$f_1(n) = n \quad 9$$

$$f_2(n) = 500n \quad 10 =$$

$$f_3(n) = \sqrt{n} \quad 6$$

$$f_4(n) = \log(\sqrt{n}) \quad 4$$

$$f_5(n) = \sqrt{\log(n)} \quad 3$$

$$f_6(n) = 1 \quad 1$$

$$f_7(n) = 3^n \quad 15$$

$$f_8(n) = n \cdot \log(n) \quad 11$$

$$f_9(n) = \frac{n}{\log(n)} \quad 8$$

$$f_{10}(n) = 700 \quad 2 =$$

$$f_{11}(n) = \log(n) \quad 5 =$$

$$f_{12}(n) = \sqrt{9n} \quad 7 =$$

$$f_{13}(n) = 2^n \quad 14$$

$$f_{14}(n) = n^2 \quad 12$$

$$f_{15}(n) = n^3 \quad 13$$

b. (functions f_{16} through f_{30})

~~$$f_{16}(n) = \frac{n}{3}$$~~

~~$$f_{17}(n) = \sqrt[3]{n}$$~~

~~$$f_{18}(n) = n!$$~~

~~$$f_{19}(n) = \log^2(n)$$~~

~~$$f_{20}(n) = \log(n!)$$~~

~~$$f_{21}(n) = 2^{\log_2(n)}$$~~

~~$$f_{22}(n) = n^{1/\log_2(n)}$$~~

~~$$f_{23}(n) = n^{\log_2(3)}$$~~

~~$$f_{24}(n) = 2^{\sqrt{\log_2 n}}$$~~

~~$$f_{25}(n) = 2^{(\log_2 n)^2}$$~~

~~$$f_{26}(n) = n \log(n)$$~~

~~$$f_{27}(n) = 2^n$$~~

~~$$f_{28}(n) = (\log_2 n)^{\log_2 n}$$~~

~~$$f_{29}(n) = n^{1/\sqrt{\log_2(n)}}$$~~

~~$$f_{30}(n) = 2^{n/\log_2 n}$$~~

5. For each function, give a tight worst-case Θ bound and explain your reasoning. For (a), also give a best-case bound for $n \geq 1$ and an input that achieves each case. For (b)–(d), assume n is a positive integer.

a.

```
def contains(lst, target):
    for value in lst:
        if value == target:
            return True
    return False
```

b.

```
def pairs(n):
    count = 0
    for i in range(n):
        for j in range(i):
            count += 1
    return count
```

c.

```
def doubling(n):
    count = 0
    for i in range(n):
        j = 1
        while j <= n:
            count += 1
            j *= 2
    return count
```

d.

```
def repeated_search(n):
    lst = list(range(n))
    count = 0
    for i in range(n):
        if -1 not in lst:
            count += 1
    return count
```

6. Trace binary search (10 minutes)

Use inclusive bounds $low = 0$ and $high = len(lst) - 1$. At each step, $mid = (low + high) // 2$. Stop on a match; otherwise use $low = mid + 1$ or $high = mid - 1$. Stop unsuccessfully when $low > high$.

```
lst = [2, 5, 8, 12, 16, 23, 38, 56, 72]
```

(a) Search for 23.

(b) Search for 15.

For each iteration, record low , $high$, mid , $lst[mid]$, and the next action. Give the returned index, or -1 if absent. Count the number of midpoint values inspected.

Coding

In this section, it is strongly recommended that you solve the problem on paper before writing code. **You are not allowed to use `.pop()`, `.append()`, `.delete()`, `.reverse()` or any other list method besides getting and setting indices (e.g. `L[i] = 10`) for any problem.**

For this lab and all labs going forward, you are required to have code typed into an IDE to be eligible for early checkout.

1. Given a list of values (`int`, `float`, `str`, ...), write a function that reverses its order in-place. You are not allowed to create a new list. Your solution must run in $\Theta(n)$, where n is the length of the list (10 minutes).

```
def reverse_list(lst):  
    """  
    : lst type: list[]  
    : return type: None  
    """
```

Modify the function to include low and high parameters that represent the positive indices to consider. Your function should reverse the list from index low to index high, inclusively. By default, low and high will be None so these parameters are optional. If they're both None (no parameters passed), set low to 0 and high to `len(lst) - 1` just like in the previous function above.

You are not allowed to create a new list. Your solution must run in $\Theta(n)$, where n is the length of the list (5 minutes).

```
def reverse_list(lst, low = None, high = None):  
    """  
    : lst type: list[]  
    : low, high type: int  
    : return type: None  
    """
```

Example:

```
lst = [1, 2, 3, 4, 5, 6], low = 0, high = 5  
reverse_list(lst) #default, no parameters passed  
print(lst) → [6, 5, 4, 3, 2, 1]
```

```
lst = [1, 2, 3, 4, 5, 6], low = 1, high = 3  
reverse_list(lst, 1, 3)  
print(lst) → [1, 4, 3, 2, 5, 6]
```

2. Define a generator that takes in a list `lst` and returns its elements only when that next element is greater than every element seen before:

```
def new_maximums(lst)
```

Example: `lst = [3, 1, 4, 4, 2, 7, 5, 9]`

Yield values: `[3, 4, 7, 9]`

3. ****The `.pop()` list method is allowed for the following problem****

- a. Given a list of positive integers with zeros mixed in, write a function to move all zeros to the end of the list while maintaining the order of the non-zero numbers. For example, given the list `[0, 1, 0, 3, 13, 0]`, the function will modify the list to become `[1, 3, 13, 0, 0, 0]`. Your solution must be in-place and run in $\Theta(n)$, where n is the length of the list. (25 minutes)

```
def move_zeros(nums):
    """
    : nums type: list[int]
    : return type: None
    """
```

Hint: You should traverse the list with 2 pointers, both starting from the beginning. One pointer will traverse through the entire list moving every iteration, but when should the other pointer move?

- b. Modify the function to include additional parameters that specify what elements are to be moved. By default, this value should be zero so this parameter is optional. If no parameters are passed, your function should still behave as the function above. (5 minutes)
- c. Write a function that removes all instances of an element `target` from a list `lst` in $O(n)$ time. (5 minutes)

```
def remove_target_elem(lst, target):
    """
    : lst type: list[int]
    : target type: int
    : return type: None
    """
```

4. **First and last occurrence**

Implement `search_range(lst, target)`. The integer list is sorted in nondecreasing order and may contain duplicates. Return a tuple (first, last) giving the first and last indices of target. If absent, return (-1, -1).

Your solution must use $O(\log n)$ worst-case time and $O(1)$ auxiliary space. **Do not scan outward from a match, slice the list, or use a built-in search routine.(25 minutes)**

```
search_range([1, 2, 2, 2, 5, 8], 2) # (1, 3)
search_range([1, 2, 2, 2, 5, 8], 4) # (-1, -1)
search_range([7, 7, 7], 7)         # (0, 2)
search_range([], 7)                # (-1, -1)
```

```
def search_range(lst, target):
    """
        : lst type: list
        : target type: int
        : return type: tuple[int]
    """
```

Test an empty list, one match, no match, all elements equal, and a repeated block touching either end. Identify the case in which scanning across duplicates can violate the required time bound and explain why.

————— OPTIONAL: —————

5. Recall the following question from Homework 1 Question 2:

```
def shift(lst, k):
    """
        : lst type: list
        : k type: int
        : return type: None
    """
```

The function takes in a list and shifts it to the left by k steps.

ex) `shift([1, 2, 3, 4, 5, 6], 2) → [3, 4, 5, 6, 1, 2]`

In the homework, you probably solved it using the list methods, `pop()` and `insert()` to shift the list or manually shifted the list each time using an extra loop. Know that your homework solution was not linear if you used either of these methods. Since the run-time of the list methods have not been discussed at this point, **do not use any of the methods for this question.**

This time, you will attempt to solve this with run-time in mind. That is, your solution must run in $\Theta(n)$, where n is the length of the list (15 minutes).

The direction will also change so that the **shift function will shift to the right instead.**

ex) `shift([1, 2, 3, 4, 5, 6], 2) → [5, 6, 1, 2, 3, 4]`

You must use the function `reverse_list(lst, low, high)` function from part 1b to solve this problem.